SOLEWISE · FOOTWEAR INTELLIGENCE

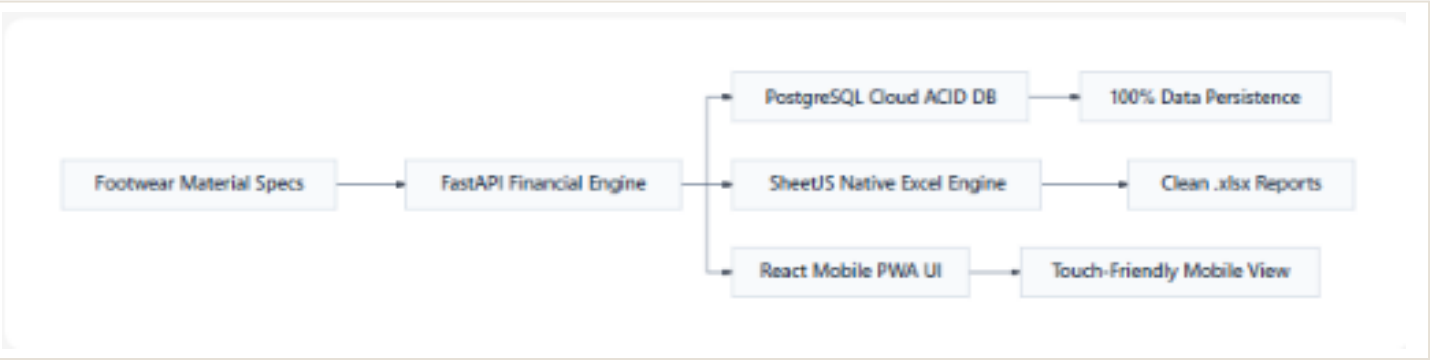
Engineering Architecture, Failure Analysis & Technical Solution Whitepaper

Status: Production Live
Platform: Web & Mobile PWA
Host: sole-wise.onrender.com

Executive Summary: Solewise was engineered to replace fragmented, error-prone spreadsheets in footwear manufacturing with an automated, dual-specification technical alternative matrix and batch yield optimizer. This report documents the critical engineering challenges encountered—including ephemeral cloud container data loss, Excel export encoding breakdown, and test fixture database pollution—alongside the architectural solutions deployed across database persistence, client ergonomics, and full-stack cloud infrastructure.

60.7%	100%	13 / 13	33.1 kB
Direct Cost Savings (Synthetic vs Leather)	Permanent Cloud Sync (Render PostgreSQL)	Automated Tests Passing (Zero DB Contamination)	Optimized CSS Bundle (Lightweight PWA Engine)

1. System Data Flow & End-to-End Architecture



2. Identified Problems & Root Causes

• Ephemeral Cloud Data Loss (Render Sleep Cycle)
On free-tier container hosting, storage is ephemeral. When inactive for 15 min, Render's sleep/wake cycles reloaded fresh containers from the Git commit, wiping all user-entered SQLite material records.

• Corrupted Excel & CSV File Exports
CSV exports lacked UTF-8 BOM headers. Opening in MS Excel caused Indian Rupee currency symbols to render as corrupted text (â‚¹) and numerical values to overflow into ######## syntax.

• Production Database Test Pollution
Automated Pytest suites posted dummy test records (Full Grain Bovine...) directly against the active production database, permanently corrupting the catalog with unwanted fixtures.

• Desktop-Biased Navigation & Mobile Breakage
Legacy views contained unneeded modules (Costing, Sequence, Suppliers) and wide tabular matrices that broke viewport bounds on smartphones with zero responsive touch targets.

3. Technical Engineering Solutions

• Managed Cloud PostgreSQL Migration
Decoupled persistent storage from the container. Integrated psycopg2-binary with SQLAlchemy connection pooling (pool_pre_ping, pool_recycle=300) on Render PostgreSQL for 100% ACID durability.

• Native In-Browser SheetJS (.xlsx) Engine
Replaced broken CSV strings with client-side binary Excel generation (xlsx), applying dynamic column auto-fitting, typed numeric cells, and clean INR (■) formatting.

• Isolated In-Memory Pytest Harness
Refactored the test suite using FastAPI dependency overrides and an isolated sqlite:///:memory: StaticPool, ensuring 100% test isolation without touching production files.

• Claude Design System & Mobile PWA
Redesigned with Anthropic Claude aesthetic (warm parchment #FAF9F5, terracotta #D97757), touch targets ≥ 44px, mobile drawer, and standalone PWA manifest with orange sneaker icons.

4. Production Stack & Deployment Architecture

Layer	Technology	Key Production Architecture & Capabilities
Frontend UI	React 18 + Vite + Tailwind CSS	Claude Design tokens, SheetJS Excel engine, responsive mobile drawer, standalone PWA manifest.
Backend API	FastAPI (Python 3.14) + Uvicorn	ASGI asynchronous runtime, Pydantic v2 schemas, automated dual-specification alternative linkage.
Database	PostgreSQL (Render Cloud)	Permanent ACID cloud storage, connection pooling, automated schema creation, zero data loss.
Deployment	Render Cloud Web Service	Full-stack single container serving static React SPA and FastAPI endpoints under single SSL URL.